

# Spring Boot Assignment

## Question 1: Explain Spring Request-Response Life Cycle

### Definition

The Spring Request-Response Life Cycle describes the complete journey of an HTTP request from the moment it hits the web server until a response is sent back to the client, passing through Spring's core components.

---

### Step-by-Step Flow

| Step | Component                       | Action                                                                                                       |
|------|---------------------------------|--------------------------------------------------------------------------------------------------------------|
| 1    | Client<br>(Browser/<br>Postman) | Sends an HTTP request (GET, POST, etc.) to the server (e.g., <code>http://localhost:8080/users/101</code> ). |
| 2    | Web Server<br>(Tomcat/Jetty)    | Receives the request and hands it over to the DispatcherServlet (the Front                                   |

|   |                   |                                                                                                                                                              |
|---|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |                   | Controller)                                                                                                                                                  |
| 3 | DispatcherServlet | Acts as the central routing hub. It consults the HandlerMapping to find which controller method should handle this request based on the URL and HTTP method. |
| 4 | HandlerMapping    | Returns the matching HandlerExecutionChain (which includes the controller method + any                                                                       |

|   |                                                  |                                                                                                                                     |
|---|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
|   |                                                  | interceptors).                                                                                                                      |
| 5 | HandlerAdapter                                   | Executes the controller method. It converts request parameters, path variables, and body into Java method arguments (data binding). |
| 6 | Controller<br>( @RestController or @Controller ) | Processes the request — calls service layer, performs business logic, and fetches/manipulates data. It returns a ModelAndView (f    |

|   |                             |                                                                                                                                                                                  |
|---|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |                             | <p>or traditional MVC) or a JSON/XML response (for REST APIs).</p>                                                                                                               |
| 7 | Interceptors (optional)     | <p><code>preHandle()</code> and <code>postHandle()</code> methods can intercept the request for cross-cutting concerns like logging, authentication, or modifying the model.</p> |
| 8 | ViewResolver (for MVC apps) | <p>Resolves the logical view name (e.g., <code>"home"</code>) to an actual</p>                                                                                                   |

|   |                |                                                                                                                                                                              |
|---|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   |                | <p>view<br/>template<br/>(e.g., <code>/WEB-INF/views/home.jsp</code> or <code>home.html</code>). For REST APIs (using <code>@ResponseBody</code>), this step is skipped.</p> |
| 9 | View Rendering | <p>The View (JSP, Thymeleaf, etc.) renders the final HTML page using data from the Model. For REST, the <code>HttpMessageConverter</code> converts the Java object to</p>    |

|    |                   |                                                                                        |
|----|-------------------|----------------------------------------------------------------------------------------|
|    |                   | JSON/XML.                                                                              |
| 10 | DispatcherServlet | Sends the final response back to the web server, which then forwards it to the client. |
| 11 | Client            | Receives the HTTP response (HTML page or JSON data).                                   |

---

## Visual Representation (Text-Based Flowchart)

text

```

Client Request
  ↓
[Web Server (Tomcat)]
  ↓
[DispatcherServlet] → HandlerMapping → identifies controller
  ↓
[HandlerAdapter] → executes controller method
  ↓
[Controller] → calls Service/DAO → returns Model+View OR JSON
  ↓

```

```
[Interceptors] (optional pre/post processing)
    ↓
[ViewResolver] (if MVC) OR [HttpMessageConverter] (if REST)
    ↓
[View Rendering] (JSP/Thymeleaf) OR JSON Serialization
    ↓
[DispatcherServlet] sends Response back
    ↓
Client receives Response
```

---

## Key Annotations Involved

- `@RequestMapping` / `@GetMapping` / `@PostMapping` – map URL to controller methods
  - `@RequestParam`, `@PathVariable`, `@RequestBody` – bind request data to method parameters
  - `@ResponseBody` – tells Spring to skip view resolution and write directly to response body
  - `@RestController` = `@Controller` + `@ResponseBody` (for REST APIs)
- 

## Example Code Snippet

java

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User user = userService.findById(id); // business logic
        return ResponseEntity.ok(user);       // returns JSON response
    }
}
```

---

## Question 2: What is the use of Maven in the Spring Boot Framework?

### Definition

Maven is a build automation and project management tool primarily used for Java projects. In Spring Boot, it serves as the backbone for dependency management, building, testing, and packaging the application.

---

### Key Uses of Maven in Spring Boot

| Use                                    | Explanati<br>on                                                                                                                                                                                                                                                                                              |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.<br>Dependen<br>cy<br>Managem<br>ent | Maven<br>handles<br>all library<br>dependen<br>cies<br>via <code>pom.xml</code><br>1 . You<br>just<br>declare a<br>starter<br>(e.g., <code>spring-boot-starter-web</code> ), and<br>Maven<br>automatic<br>ally pulls<br>all<br>transitive<br>dependen<br>cies (like<br>Spring<br>MVC,<br>Tomcat,<br>Jackson) |



|                         |                                                                                                                                                                                                                                  |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         | with compatible versions.                                                                                                                                                                                                        |
| 2. Spring Boot Starters | <p>Maven resolves "starter" parent POMs (e.g., <code>spring-boot-starter-parent</code>) which provide pre-configured dependency versions, plugin configurations, and default settings — eliminating version mismatch issues.</p> |
| 3. Build & Packaging    | Maven compiles                                                                                                                                                                                                                   |

|                                                    |                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                    | <p>the code,<br/>runs tests,<br/>and<br/>packages<br/>the<br/>applicatio<br/>n into<br/>a JAR (or<br/>WAR) file<br/>using<br/>command<br/>s like <code>mvn</code><br/><code>clean</code><br/><code>package</code> .<br/>The built<br/>JAR is fat/<br/>über<br/>JAR conta<br/>ining all<br/>dependen<br/>cies,<br/>making it<br/>self-<br/>executabl<br/>e.</p> |
| <p>4.<br/>Running<br/>the<br/>Applicatio<br/>n</p> | <p>Using <code>mvn</code><br/><code>spring-<br/>boot:run</code> ,<br/>you can<br/>start the<br/>Spring<br/>Boot<br/>applicatio</p>                                                                                                                                                                                                                             |

|                     |                                                                                                                                                                                                                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                     | <p>n directly without manually packaging it — ideal for development.</p>                                                                                                                                                                                                             |
| 5. Plugin Ecosystem | <p>Maven integrates with Spring Boot's own plugin ( <code>spring-boot-maven-plugin</code> ) to:</p> <ul style="list-style-type: none"> <li>• Create executable JARs</li> <li>• Repackage existing JARs</li> <li>• Exclude provided dependencies</li> <li>• Set main class</li> </ul> |

|                            |                                                                                                                                                                                         |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            | attributes<br>in the<br>manifest                                                                                                                                                        |
| 6. Testing                 | Maven runs unit and integration tests automatically during the <code>test</code> and <code>verify</code> phases using JUnit and Mockito dependencies declared in <code>pom.xml</code> . |
| 7. Project Standardization | Maven enforces a standard project structure ( <code>src/main/java</code> , <code>src/test/java</code> , <code>src/main/resources</code> ),                                              |

|                            |                                                                                                                                                                             |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            | making it easier for teams to collaborate and for CI/CD tools to understand the project.                                                                                    |
| 8.<br>Lifecycle Management | Maven provides a well-defined build lifecycle (clean, compile, test, package, install, deploy), allowing seamless integration with build servers (Jenkins, GitHub Actions). |
| 9.<br>Repository           | Maven download                                                                                                                                                              |

|                     |                                                                                                                                                                                                                             |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| y<br>Managem<br>ent | s<br>dependen<br>cies from<br>central<br>repositorie<br>s (Maven<br>Central,<br>Spring<br>Milestone<br>s, etc.).<br>You can<br>also<br>configure<br>private/cor<br>porate<br>repositorie<br>s for<br>internal<br>libraries. |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

## Minimal `pom.xml` Example for Spring Boot

xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
```

```

<version>1.0.0</version>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

## Common Maven Commands Used with Spring Boot

| Command                | Purpose                                                             |
|------------------------|---------------------------------------------------------------------|
| <code>mvn clean</code> | Deletes the <code>target</code> / folder (removes compiled classes) |

|                                  |                                                                                    |
|----------------------------------|------------------------------------------------------------------------------------|
|                                  | and old JARs)                                                                      |
| <code>mvn compile</code>         | Compiles the source code                                                           |
| <code>mvn test</code>            | Runs unit tests                                                                    |
| <code>mvn package</code>         | Creates the JAR/WAR file in <code>target /</code>                                  |
| <code>mvn install</code>         | Installs the built JAR into the local Maven repository (for use in other projects) |
| <code>mvn spring-boot:run</code> | Runs the Spring Boot application directly                                          |



|                         |            |
|-------------------------|------------|
| <code>mvn</code>        | Displays   |
| <code>dependency</code> | the        |
| <code>:tree</code>      | complete   |
|                         | dependen   |
|                         | cy tree    |
|                         | (helps     |
|                         | debug      |
|                         | version    |
|                         | conflicts) |

---

## Why Maven Over Manual Management?

- Without Maven, you'd manually download JARs, add them to classpath, and handle version conflicts — which is error-prone and time-consuming.
- Maven automates all of this, letting you focus on writing business logic rather than infrastructure setup.

---

## Conclusion

Maven is essential in Spring Boot because it manages the entire project lifecycle — from dependency resolution to packaging and deployment. Together with Spring Boot's opinionated defaults, Maven drastically reduces configuration overhead and accelerates development.

---

## Submission Instructions